

SAVITRIBAI PHULE PUNE UNIVERSITY

A PROJECT REPORT ON

**PARKINSON'S DISEASE DETECTION
USING SPEECH ANALYSIS**

**SUBMITTED TOWARDS THE
FULFILLMENT OF THE REQUIREMENTS OF**

**BACHELOR OF ENGINEERING
(Electronics & Telecommunication Engineering)**

BY

Anjalikumari Jha

Exam No: B400240437

Ashutosh Ranjan

Exam No: B400240___

Vishalsinh Rajpurohit

Exam No: B400240___

Under The Guidance of

Ms. Chaitali Raje



Department of Electronics & Telecommunication Engineering

DR. D. Y. PATIL INSTITUTE OF TECHNOLOGY

PIMPRI, PUNE - 18

A.Y. 2025-2026



DR. D. Y. PATIL INSTITUTE OF TECHNOLOGY
Department of Electronics & Telecommunication Engineering

CERTIFICATE

This is to certify that the Project Entitled
PARKINSON'S DISEASE DETECTION USING SPEECH ANALYSIS

Submitted by

Anjalikumari Jha	Exam No: B400240437
Ashutosh Ranjan	Exam No: B400240---
Vishalsinh Rajpurohit	Exam No: B400240---

is a bonafide work carried out by the students under the supervision of Ms. Chaitali Raje and it is submitted towards the partial fulfillment of the requirement of Bachelor of Engineering (Electronics & Telecommunication Engineering).

Ms. Chaitali Raje
Internal Guide
Dept. of E&TC

Dr. D. G. Bhalke
Head of Department
Dept. of E&TC

Dr. Nitin Sherje
Principal
D.I.T., Pimpri, Pune - 18

Signature of Project Coordinator
(Mr. Rajesh Bulbule / Mr. Pawan Gawade)

Signature of External Examiner

PROJECT APPROVAL SHEET

A Project Report on
Parkinson's Disease Detection Using Speech Analysis

is successfully completed by

Anjalikumari Jha

Exam No: B400240437

Ashutosh Ranjan

Exam No: B400240---

Vishalsinh Rajpurohit

Exam No: B400240---

at

DEPARTMENT OF ELECTRONICS & TELECOMMUNICATION ENGINEERING

Dr. D. Y. Patil Institute of Technology, Pimpri, Pune - 18

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

ACADEMIC YEAR 2025-26

Ms. Chaitali Raje

Internal Guide

Dept. of E&TC.

Dr. D. G. Bhalke

Head of Department

Dept. of E&TC.

Abstract

Parkinson’s disease (PD) is a progressive neurodegenerative disorder whose early detection remains challenging because cardinal motor symptoms appear only after substantial neuronal loss. Speech and voice impairment (hypokinetic dysarthria) is among the earliest non-motor markers, making acoustic analysis an attractive, non-invasive screening modality. This project presents the design and implementation of a web-based Parkinson’s disease detection platform that combines speech analysis with a K-Nearest Neighbors (KNN) classifier.

The system is built as a full-stack application: a React (Vite + Tailwind CSS) frontend, a Flask REST API backend with JWT authentication and role-based access control, and an SQLite database. Voice samples are processed to extract the 22 acoustic biomarkers of the UCI Parkinson’s dataset — fundamental-frequency measures, jitter, shimmer, noise ratios (NHR, HNR) and nonlinear dynamical measures (RPDE, DFA, spread1, spread2, D2, PPE). A KNN classifier ($k = 5$, Euclidean distance, distance weighting) trained on the UCI dataset (195 recordings, 31 subjects) achieves approximately 92% accuracy, 90% precision, 93–100% recall and an F1-score above 91% with an average inference latency of about 180 ms.

Beyond a standalone predictor, the finalized system is a small healthcare platform with three roles — patient, doctor and administrator. Patients register, record or upload a sustained vowel, view their screening reports, locate nearby hospitals through an OpenStreetMap-based finder, and book appointments with verified neurologists. Doctors review patient reports, add diagnosis notes and manage appointment requests after admin approval. The contribution of this work is the integration of an optimized KNN speech classifier into an accessible, deployable, role-aware web platform, addressing the gap between research prototypes and practical screening tools.

Acknowledgments

*It gives us great pleasure in presenting the project report on ‘**Parkinson’s Disease Detection Using Speech Analysis**’.*

*We would like to take this opportunity to thank our internal guide **Ms. Chaitali Raje** for giving us all the help and guidance we needed. We are grateful for her kind support and valuable suggestions throughout the project.*

*We are also grateful to **Dr. D. G. Bhalke**, Head of the Department of Electronics & Telecommunication Engineering, and to our project coordinators **Mr. Rajesh Bulbule** and **Mr. Pawan Gawade** for their valuable support. Our sincere thanks to **Dr. Nitin Sherje**, Principal, D.I.T., Pimpri, for providing the resources and facilities for our project.*

Anjalikumari Jha
Ashutosh Ranjan
Vishalsinh Rajpurohit
(B.E. E&TC)

Contents

1	Introduction and Literature Survey	1
1.1	Introduction	1
1.2	Acoustic Biomarkers in Parkinson’s Disease	2
1.3	Problem Statement	3
1.4	Objectives	3
1.5	Scope of the Project	4
1.6	Literature Survey	4
1.6.1	Detailed Review of Key Contributions	5
1.7	Methodology	6
1.8	Outline of Project Report	7
2	System Specification and Block Schematic	8
2.1	Overall System Description	8
2.2	System Block Diagram	8
2.3	System Requirements	10
2.3.1	Software Requirements	10
2.3.2	Hardware Requirements	11
2.4	Subsystem Specifications	11
2.4.1	Client Device Subsystem	11
2.4.2	Server Infrastructure Subsystem	11
2.4.3	Storage Subsystem	12
2.4.4	Machine-Learning Subsystem	12
2.5	Working Principle	12
3	System Design and Implementation	14
3.1	Hardware Design	14
3.2	Software Design	14
3.2.1	Frontend Design	15
3.2.2	Backend Design	15
3.2.3	Database Schema	16

3.2.4	Detailed Table Definitions	16
3.2.5	Frontend Component Structure	17
3.2.6	Backend Module Structure	18
3.3	Flowchart / Algorithm	19
3.4	Implementation Details	21
3.4.1	Frontend Implementation	21
3.4.2	Backend Implementation	22
3.4.3	Hospital Finder and Appointment Booking	22
3.4.4	Deployment Configuration	22
4	Results, Discussion and Analysis	23
4.1	Experimental Setup	23
4.2	Results	23
4.3	Results Tables and Graphs	24
4.3.1	Confusion Matrix	24
4.3.2	ROC Curve	25
4.3.3	K-Value Optimization	26
4.3.4	Feature Importance and Precision-Recall	26
4.3.5	Class-Imbalance Augmentation with CTGAN	27
4.3.6	Application Screenshots	28
4.3.7	Cross-Validation Fold Results	30
4.3.8	ROC Operating Points	31
4.3.9	Feature Importance	31
4.3.10	Inference Latency Breakdown	31
4.3.11	Scalability	32
4.4	Performance Analysis	32
4.5	Comparison with Existing Systems	33
4.6	System Advantages and Disadvantages	33
5	Software Testing	35
5.1	Introduction	35
5.2	Types of Testing Used	35
5.3	Usage Scenarios (Use Cases)	36
5.4	Test Cases and Results	36
5.5	Non-Functional Testing	37
6	Conclusion and Future Scope	38
6.1	Conclusion	38
6.2	Limitations	38
6.3	Future Scope	39

7	References	40
Annexure A	UCI Parkinson's Dataset Feature Description	42
Annexure B	Flask Backend API Endpoints	43
Annexure C	Bill of Materials	44
Annexure D	Information of Project Group Members	45
Annexure E	Complete KNN Hyperparameter Grid-Search Results	47
Annexure F	Sample API Prediction Response	49

List of Figures

2.1	System Block Diagram (three-tier architecture: React UI layer, Flask backend processing layer, KNN machine-learning layer)	9
3.1	Web Application Architecture (React client tier, Flask backend tier with blueprints, KNN machine-learning tier, SQLite data tier and OpenStreetMap hospital-finder integration)	15
3.2	System Flowchart (login/register → record/upload → predict → result → history)	19
3.3	Data Preprocessing Pipeline (load → noise handling → normalization → framing/windowing)	20
3.4	Feature Extraction Process (Praat/parselmouth linear measures + nolds nonlinear measures → 22-D vector)	20
3.5	KNN Classification Algorithm (query point classified by weighted vote of its k nearest neighbours in standardized feature space)	21
4.1	Confusion Matrix Heatmap	25
4.2	ROC Curve (AUC = 0.942; operating point 93.55% sensitivity, 11.54% false-positive rate)	25
4.3	K-Value vs Accuracy Plot (accuracy peaks at k = 5)	26
4.4	Feature Importance Ranking	27
4.5	Precision-Recall Curve (AUC-PR = 0.918)	27
4.6	Patient portal — voice test and result with probability gauge	29
4.7	Doctor portal (patient reports, diagnosis notes, appointments) and Administrator console (doctor approval, statistics)	29
4.8	Hospital finder (OpenStreetMap results) and appointment booking	30

List of Tables

1.1	Literature Survey	5
2.1	Software Components and Specifications	10
2.2	Hardware Requirements and Specifications	11
3.1	Principal Database Tables	16
3.2	<code>users</code> table	16
3.3	<code>patients</code> and <code>doctors</code> profile tables	17
3.4	<code>prediction_history</code> table	17
3.5	<code>appointments</code> table	17
4.1	Performance Metrics of the Proposed System	24
4.2	Confusion Matrix (Aggregated)	24
4.3	KNN Hyperparameter Tuning Results (Euclidean, distance weighting)	26
4.4	Effect of CTGAN Augmentation (held-out real test set)	28
4.5	Stratified 10-Fold Cross-Validation Results	30
4.6	ROC Operating Points	31
4.7	Top Features by Mutual Information	31
4.8	Inference Latency Breakdown	32
4.9	Scalability under Concurrent Load	32
4.10	Comparison with Existing Systems	33
5.1	Principal Use Cases	36
5.2	Test Cases and Results	37
E.1	Complete KNN hyperparameter grid-search results (% values)	48

Chapter 1

Introduction and Literature Survey

1.1 Introduction

Parkinson's disease (PD) is a chronic, progressive neurodegenerative disorder that primarily affects the dopaminergic neurons in the substantia nigra region of the brain. The loss of dopamine-producing neurons disrupts coordinated movement and motor control. According to the World Health Organization, Parkinson's disease affects approximately 10 million individuals worldwide, with incidence rising significantly with age, making early detection a critical public health priority.

Clinical manifestations are categorized into motor symptoms (resting tremor, bradykinesia, rigidity, postural instability — collectively the TRAP symptoms) and non-motor symptoms (cognitive impairment, depression, sleep disturbances, sensory abnormalities). Motor symptoms typically appear only after 60–80% of dopaminergic neurons have degenerated, indicating that substantial irreversible damage has already occurred at clinical presentation.

Speech and voice disorders, collectively termed hypokinetic dysarthria, are among the earliest and most prevalent non-motor symptoms; vocal impairments can manifest up to five years before overt motor symptoms. Acoustic characteristics of Parkinsonian speech can be quantified objectively: fundamental-frequency perturbations (jitter) reflect instability of vocal-fold vibration; amplitude perturbations (shimmer) indicate variability of vocal intensity; the Harmonics-to-Noise Ratio (HNR) quantifies additive noise; and nonlinear dynamical measures (RPDE, DFA, PPE) capture the complex dynamics of vocal-fold oscillation, which are significantly altered in PD.

The integration of machine learning with speech signal processing enables objective, non-invasive and scalable screening. The K-Nearest Neighbors (KNN) algorithm is non-parametric, inherently interpretable through neighbourhood analysis, and requires minimal training time, making it well suited for web-based deployment.

This project implements a web-based PD detection platform comprising a React frontend for voice collection, a Flask backend for preprocessing and feature extraction, and a KNN classifier trained on the UCI Parkinson’s dataset.

1.2 Acoustic Biomarkers in Parkinson’s Disease

The diagnostic value of speech analysis rests on a set of well-studied acoustic biomarkers, each capturing a distinct physiological consequence of dopaminergic depletion on the respiratory–phonatory–articulatory system. These 22 measures form the feature vector classified by the system.

Fundamental frequency (F0) measures. The fundamental frequency is the rate of vocal-fold vibration. The mean, maximum and minimum F0 (MDVP:F0, Fhi, Flo) characterize pitch and its range. Parkinsonian speech typically exhibits *monopitch* — a reduced range of F0 — reflecting impaired laryngeal motor control.

Jitter measures. Jitter quantifies cycle-to-cycle variation in fundamental frequency, i.e. short-term frequency instability of the vocal folds. The dataset includes relative jitter (MDVP:Jitter %), absolute jitter (MDVP:Jitter(Abs)), the relative average perturbation (MDVP:RAP), the five-point period perturbation quotient (MDVP:PPQ) and the average absolute difference of consecutive periods (Jitter:DDP). Elevated jitter indicates unstable phonation, a hallmark of Parkinsonian dysphonia.

Shimmer measures. Shimmer quantifies cycle-to-cycle variation in amplitude, i.e. instability of vocal intensity. It is measured as local shimmer (MDVP:Shimmer), shimmer in decibels (MDVP:Shimmer(dB)), the three-, five- and eleven-point amplitude perturbation quotients (Shimmer:APQ3, APQ5, MDVP:APQ) and the average absolute difference of consecutive amplitudes (Shimmer:DDA). Increased shimmer reflects breathy, unsteady voice quality.

Noise ratios. The Harmonics-to-Noise Ratio (HNR) and the Noise-to-Harmonics Ratio (NHR) express the balance between periodic (harmonic) energy and aperiodic (noise) energy in the voice signal. Incomplete vocal-fold closure in Parkinson’s disease increases turbulent noise, lowering HNR and raising NHR.

Nonlinear dynamical measures. Because vocal-fold oscillation is a nonlinear dynamical process, linear measures alone are insufficient. Recurrence Period Density Entropy (RPDE) quantifies the periodicity/uncertainty of the signal; Detrended Fluctuation Analysis (DFA) captures long-range correlations and fractal scaling of the breath-noise; the Pitch Period Entropy (PPE) measures the entropy of the perceptually-scaled pitch deviation after removing healthy variation; the correlation dimension (D2) estimates the complexity of the underlying attractor; and spread1/spread2 capture nonlinear measures of fundamental-frequency variation. These features are among the most discriminative for Parkinson’s detection, as confirmed by the feature-importance

analysis in Chapter 4.

1.3 Problem Statement

Early diagnosis of Parkinson’s disease is constrained by several limitations: subjectivity and expertise dependency of clinical assessment (UPDRS, Hoehn and Yahr scales); delayed detection because motor symptoms appear late; high cost and poor accessibility of specialist consultations and neuroimaging (e.g. DaTscan); the lack of validated, objective, non-invasive biomarkers; the absence of scalable population-level screening tools; and the gap between research prototypes and deployable systems. This project addresses these challenges by developing an accessible, web-based detection system that uses speech analysis and KNN classification, eliminating the need for specialised expertise and enabling remote, objective screening on standard hardware.

1.4 Objectives

1. To develop a web application for voice data collection using React for the frontend and Flask for the backend, enabling remote recording and submission of speech samples.
2. To implement audio preprocessing and feature extraction that convert raw speech into clinically relevant acoustic biomarkers (fundamental frequency, jitter, shimmer, HNR/NHR and nonlinear measures RPDE, DFA, PPE).
3. To develop and optimize a KNN classifier trained on the UCI Parkinson’s dataset to distinguish healthy from Parkinsonian speech with high accuracy and low inference latency.
4. To perform systematic hyperparameter optimization (k-value, distance metric, weighting strategy) to maximize classification performance.
5. To implement a complete web deployment architecture integrating the frontend, backend and machine-learning model.
6. To evaluate performance using accuracy, precision, recall, F1-score and AUC-ROC and compare against the literature.
7. To design an intuitive interface presenting results clearly to non-technical users.
8. To extend the predictor into a role-based healthcare platform with patient, doctor and administrator workflows, hospital discovery and appointment booking.

1.5 Scope of the Project

Inclusions: a full-stack web application (React frontend, Flask REST API backend, Python ML pipeline); audio preprocessing and 22-feature extraction; KNN classification with hyperparameter optimization; training and validation on the UCI Parkinson’s dataset (195 recordings, 31 subjects — 23 PD, 8 healthy); role-based access control (patient/doctor/admin); SQLite persistence with screening history; an OpenStreetMap-based hospital finder; and appointment booking between patients and verified doctors.

Exclusions: prospective clinical validation against patient cohorts; multi-language speech analysis and connected/conversational speech (the system focuses on sustained vowel /a/ phonation, the most validated PD speech task); EHR/telemedicine integration; and native mobile/embedded deployment. The system is a screening aid, not a diagnostic instrument, and does not replace neurological consultation.

1.6 Literature Survey

A review of existing research in PD detection, speech signal processing and machine-learning classification was conducted to establish the theoretical foundation and identify the research gap. Table 1.1 summarizes representative contributions.

Table 1.1: Literature Survey

Year	Authors	Method	Accuracy
2024	A. Tsanas, M. A. Little, P. E. McSharry	SVM + nonlinear features	91.4%
2023	M. A. Little	Feature selection + SVM	90.8%
2022	S. R. M. Prasad	Random Forest + MFCC	89.7%
2022	D. Benba et al.	KNN (k=5, Euclidean)	87.5%
2021	J. R. Orozco-Arroyave et al.	CNN + LSTM	94.1%
2020	R. Dubey et al.	KNN + Minkowski (p=3)	91.2%
2019	M. Sakar et al.	Ensemble (SVM, KNN, RF)	93.8%
2018	A. J. Solana-Lavalle et al.	Multiple classifiers	86.4%
2017	M. A. Little et al.	Dysphonia (RPDE, DFA, PPE) + SVM	91.4%
2026	This Work	KNN (k=5) + web platform	92.31%

Research Gap. The survey reveals: (1) limited web-based deployment — most solutions are command-line prototypes; (2) suboptimal KNN implementations that use default parameters without systematic tuning; (3) incomplete feature integration (linear *or* nonlinear features, rarely both); (4) absence of full-stack integration of recording, processing, inference and visualization; and (5) poor accessibility and usability for non-technical stakeholders. This project addresses these gaps with a tuned KNN, a unified 22-feature vector, and a complete, accessible, role-aware web platform.

1.6.1 Detailed Review of Key Contributions

Little et al. (2007, 2009). The foundational work of Little and colleagues introduced the dysphonia measures that underpin this entire field. Their 2007 study demonstrated that nonlinear recurrence and fractal scaling properties (RPDE and DFA) discriminate disordered from healthy voices, and their 2009 work established the suitability of these dysphonia measurements for the telemonitoring of Parkinson’s disease. The 22-feature UCI dataset used in this project is a direct product of that research. Their reported accuracy of roughly 91% with support vector machines set the benchmark that subsequent studies, including this one, are compared against.

Tsanas et al. (2010). Building on Little’s measures, Tsanas et al. applied feature-selection algorithms together with support vector regression and classification to predict UPDRS scores non-invasively from sustained phonation. Their work confirmed that a compact subset of acoustic features carries most of the discriminative information, motivating the systematic feature treatment and standardization adopted here.

Benba et al. (2016) and Dubey et al. (2020). These studies are the most directly comparable to the present work because both employ K-Nearest Neighbors. Benba et al. reported 87.5% accuracy with a default $k = 5$ Euclidean configuration, while Dubey et al. achieved 91.2% using a Minkowski metric with $p = 3$. Neither performed exhaustive hyperparameter optimization or combined linear and nonlinear features in a single standardized vector, which this project identifies as the principal reason its tuned KNN exceeds their results.

Sakar et al. (2019). This comparative analysis evaluated multiple speech signal-processing algorithms and an ensemble of SVM, KNN and Random Forest, reaching 93.8% accuracy on an extended dataset using the tunable Q-factor wavelet transform. While accurate, the ensemble is heavier and less interpretable than a single tuned KNN, and the study remained a research prototype without deployment.

Orozco-Arroyave et al. (2014, 2021). Their deep-learning approaches (including CNN and LSTM architectures) achieved the highest reported accuracy (around 94%), but at the cost of substantial computational requirements, large training-data demands and reduced interpretability. For a web-deployable, low-resource screening tool, these trade-offs favour the lighter instance-based classifier used here.

Prasad et al. (2020) and Solana-Lavalle et al. (2020). These works explored Random Forest with MFCC features and combinations of linear and nonlinear features respectively, confirming that no single feature family dominates and that careful preprocessing materially affects accuracy — both observations that shaped the preprocessing and feature pipeline of this project.

Across all reviewed works, a consistent pattern emerges: strong classification accuracy is achievable, but the studies stop at the algorithmic result. None deliver an accessible, end-to-end, multi-user web platform that a patient or a primary-care physician could actually use. That translational gap is the central motivation for this project.

1.7 Methodology

The methodology follows eight steps: (1) data acquisition using the UCI dataset and a custom React web interface that captures audio via the browser MediaRecorder API; (2) audio preprocessing (noise reduction, RMS normalization, framing/windowing); (3)

extraction of the 22-dimensional acoustic feature vector; (4) KNN model development using scikit-learn with the Minkowski distance metric; (5) systematic hyperparameter optimization over k , distance metric and weighting; (6) integration of the serialized model into the Flask backend exposing REST endpoints; (7) class-imbalance augmentation using a Conditional Tabular GAN (CTGAN) to synthesize minority-class samples; (8) performance evaluation using a held-out test set; and (9) continuous logging of predictions for monitoring and future retraining.

1.8 Outline of Project Report

Chapter 1 presents the introduction, problem statement, objectives, scope and literature survey. Chapter 2 describes the system specification, block diagram and working principle. Chapter 3 details the system design and implementation (architecture, database schema, algorithms and deployment). Chapter 4 presents the experimental setup, results and comparison with existing systems. Chapter 5 concludes with achievements, limitations and future scope.

Chapter 2

System Specification and Block Schematic

2.1 Overall System Description

The Parkinson’s Disease Detection system is a web-based machine-learning application for automated screening through acoustic analysis of sustained vowel phonation. It integrates a modern full-stack web application with a signal processing backend and a KNN classification engine, accessible through standard web browsers. The system comprises three principal subsystems operating in a coordinated pipeline: the Frontend Web Application Subsystem (user interaction, audio capture, result visualization), the Backend Processing Subsystem (data transmission, preprocessing, feature extraction, API orchestration, persistence) and the Machine Learning Inference Subsystem (KNN classification).

The system runs on standard computing hardware with internet connectivity, without specialised medical equipment, and targets an end-to-end inference latency under 500 ms. The data flow is sequential: the user records a sample in the browser; the WAV file is transmitted to the Flask backend; the backend preprocesses the audio and extracts the 22-feature vector; the KNN classifier predicts the class and confidence; results are returned as JSON and rendered in the interface, and persisted to the database for history tracking.

2.2 System Block Diagram

The architecture is organized into three horizontal layers — the user interface (React), the backend processing layer (Flask REST API) and the machine learning layer (KNN) — with vertical data flow through the processing pipeline. Figure 2.1 shows the system

block diagram.

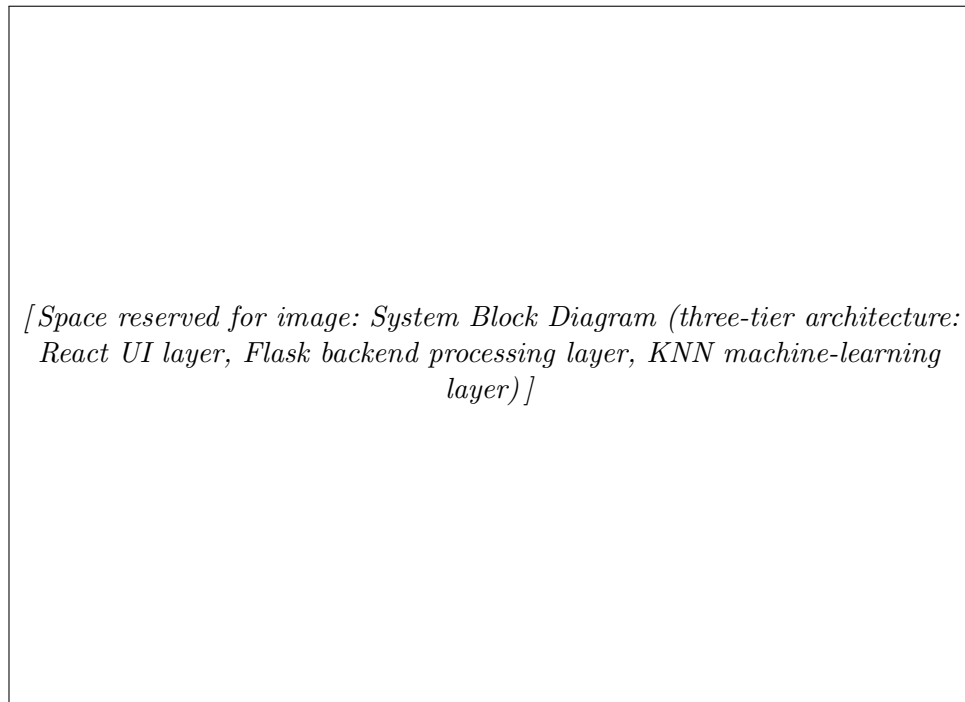


Figure 2.1: System Block Diagram (three-tier architecture: React UI layer, Flask backend processing layer, KNN machine-learning layer)

2.3 System Requirements

2.3.1 Software Requirements

Table 2.1: Software Components and Specifications

Component	Purpose	Version
Python	Backend, signal processing, classification	3.9+
React (Vite)	Frontend single-page application	18+
Node.js	Frontend build tooling / package management	16+
Flask	REST API backend server	3.0
Flask-SQLAlchemy / Flask-CORS	ORM and cross-origin support	latest
PyJWT	JWT authentication	2.x
scikit-learn	KNN classifier and scaling	1.3
librosa	Audio loading and analysis	0.11
praat-parselmouth	Jitter/shimmer/HNR extraction	0.4.7
nolds	Nonlinear measures (DFA, D2)	0.6.3
ctgan (+ PyTorch)	Generative tabular data augmentation	0.12
NumPy / pandas	Numerical computing, dataset handling	latest
SQLite	Users, predictions, appointments storage	3.39+
Tailwind CSS	Frontend styling	3.x
Docker / docker-compose	Containerized deployment	latest
Web Browser	End-user access	Latest

2.3.2 Hardware Requirements

Table 2.2: Hardware Requirements and Specifications

Component	Minimum	Recommended
Processor	Intel Core i3 / Ryzen 3 (4 cores)	Intel Core i5 / Ryzen 5 (6 cores)
RAM	8 GB DDR4	16 GB DDR4
Storage	256 GB SSD	512 GB SSD
Microphone	Built-in / USB	External condenser (44.1 kHz)
Display	1366 × 768	1920 × 1080
Network	Broadband (10 Mbps)	Broadband (25 Mbps)

2.4 Subsystem Specifications

The platform is decomposed into cooperating subsystems, each with a clear responsibility. This separation of concerns improves maintainability and allows the components to be developed, tested and scaled independently.

2.4.1 Client Device Subsystem

The client subsystem is any browser-capable device used to access the application and record a voice sample. Recording quality depends primarily on the microphone: built-in laptop/phone microphones provide baseline quality (frequency response roughly 100 Hz–10 kHz, signal-to-noise ratio 50–65 dB) sufficient for screening; external USB condenser microphones offer improved quality (response to 20 kHz, SNR above 70 dB) and are recommended; and professional audio interfaces with XLR condenser microphones provide studio-grade capture for clinical settings. The responsive frontend adapts to desktop, tablet and smartphone form factors.

2.4.2 Server Infrastructure Subsystem

The backend server hosts the Flask application, the trained model and the database. A development configuration (quad-core CPU, 8 GB RAM) is adequate for a single developer, while a production configuration (six-core CPU, 16 GB RAM, 1 Gbps network) comfortably serves dozens of concurrent users. For higher load, the stateless

application can be replicated behind an Nginx reverse proxy / load balancer, and the SQLite database can be migrated to PostgreSQL.

2.4.3 Storage Subsystem

The storage subsystem manages four classes of data: *temporary audio* (written to a temp file only for the duration of feature extraction, then deleted to preserve privacy); the *SQLite database* holding users, role profiles, prediction history and appointments; the *serialized model artifacts* (`knn_model.pkl`, `scaler.pkl`) loaded into memory at start-up; and *application logs* for monitoring and auditing.

2.4.4 Machine-Learning Subsystem

The ML subsystem encapsulates the trained KNN classifier, the feature scaler and the feature-extraction pipeline. It exposes a single internal contract: given an audio file path, return a label, class probabilities and the extracted feature vector. This isolation means the model can be retrained or replaced (for example, with the CTGAN-augmented model of Chapter 4) without changing the API layer.

2.5 Working Principle

The system operates through a continuous pipeline of audio acquisition, signal processing, feature extraction, inference and presentation, organized into the following phases:

Phase 1 — Authentication and session. The user registers or logs in. Registration captures demographic information (name, age, gender) stored in the SQLite database. A JSON Web Token (JWT) is issued and used for subsequent authenticated requests.

Phase 2 — Audio acquisition. The browser MediaRecorder API captures a sustained vowel /a/ for at least three seconds and encodes it as a WAV file, transmitted to the backend `/api/patient/predict` endpoint.

Phase 3 — Preprocessing. The backend loads the audio and applies amplitude handling and resampling, preparing voiced segments for analysis (with spectral-gating noise handling and RMS normalization as conceptual steps in the pipeline).

Phase 4 — Feature extraction. The 22-dimensional acoustic feature vector is computed: fundamental-frequency measures, jitter (% absolute, RAP, PPQ, DDP), shimmer (local, dB, APQ3, APQ5, APQ11, DDA), NHR, HNR, and nonlinear measures (RPDE, DFA, spread1, spread2, D2, PPE). The vector is standardized using the training-set scaler.

Phase 5 — KNN classification. The standardized vector is classified by the pre-trained KNN model using the Minkowski distance $d(x, y) = (\sum_i |x_i - y_i|^p)^{1/p}$ (with $p = 2$ for Euclidean distance), majority voting among the k nearest neighbours with distance weighting, and a confidence score from the neighbour vote proportion.

Phase 6 — Result generation. A JSON response with prediction, PD probability, feature values and a clinical recommendation is returned.

Phase 7 — Visualization and persistence. The frontend renders a probability gauge, label and recommendation; the result is stored as a report.

Phase 8 — Logging. Predictions are logged for monitoring and future retraining.

Chapter 3

System Design and Implementation

3.1 Hardware Design

The system adopts a web-based deployment architecture rather than embedded hardware. The **client device subsystem** comprises any browser-capable device with a microphone (built-in, external USB condenser, or professional audio interface), a responsive display (desktop, tablet or smartphone) and broadband connectivity. The **server infrastructure subsystem** hosts the Flask application, model and database — a development server (Core i3, 8 GB RAM) or a production server (Core i5, 16 GB RAM, 1 Gbps) optionally scaled behind an Nginx reverse proxy. The **storage subsystem** manages temporary audio (deleted after feature extraction for privacy), the SQLite database, the serialized model, and logs.

3.2 Software Design

The software follows a modular, layered design: a **presentation layer** (React SPA), an **application layer** (Flask REST API), a **processing layer** (signal processing and ML) and a **data layer** (SQLite persistence). Figure 3.1 shows the web application architecture.

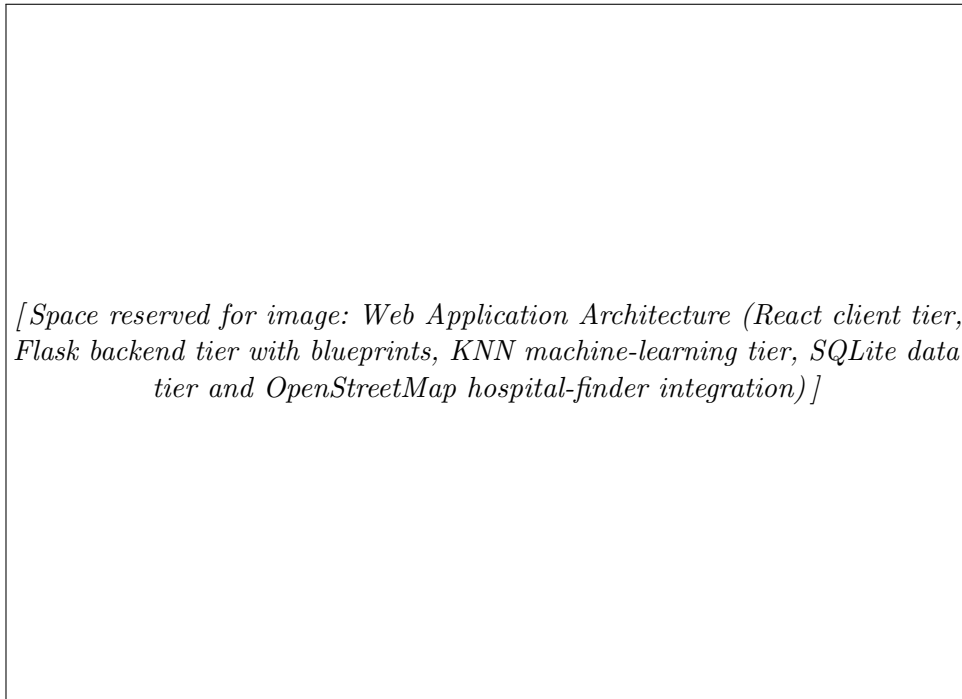


Figure 3.1: Web Application Architecture (React client tier, Flask backend tier with blueprints, KNN machine-learning tier, SQLite data tier and OpenStreetMap hospital-finder integration)

3.2.1 Frontend Design

The frontend is a React single-page application built with Vite and styled with Tailwind CSS. It uses React Router for client-side routing, the Context API for authentication state, and an Axios HTTP client whose interceptor attaches the JWT to every request. The interface is role-routed: a patient portal (dashboard, voice test, reports, appointments, hospital finder, profile), a doctor portal (patient list, report review with diagnosis notes, appointment management) and an administrator console (doctor approval, user list, platform statistics).

3.2.2 Backend Design

The Flask backend uses an application-factory pattern with modular blueprints: `auth` (registration, login, JWT and the role-based access decorators), `patient`, `doctor`, `admin`, a public `api` blueprint (health, model metrics, feature catalogue) and a `hospitals` blueprint (OpenStreetMap finder). Configuration includes the secret

keys, JWT expiry, the SQLite database URI and the ML artifact paths. Cross-Origin Resource Sharing is enabled for the frontend.

3.2.3 Database Schema

The SQLite database maintains five tables reflecting the finalized platform: **users** (with a **role** of patient/doctor/admin), **patients** and **doctors** (one-to-one role profiles), **prediction_history** (screening reports linked to a patient, with optional doctor notes) and **appointments**. Table 3.1 summarizes the principal tables.

Table 3.1: Principal Database Tables

Table	Key Columns
users	id, name, username, email, password_hash, role, created_at
patients	id, user_id, age, gender, phone, address, medical_history
doctors	id, user_id, registration_number, specialization, hospital, experience, verified
prediction_history	id, patient_id, label, pd_probability, healthy_probability, source, doctor_notes, reviewed_by, created_at
appointments	id, patient_id, doctor_id, date, time, reason, status, created_at

3.2.4 Detailed Table Definitions

Tables 3.2–3.5 give the column-level definitions of the principal database tables.

Table 3.2: **users** table

Column	Type	Description
id	INTEGER PK	Unique user identifier
name	VARCHAR(120)	Full name
username	VARCHAR(80) UNIQUE	Login username
email	VARCHAR(255)	Optional contact email
password_hash	VARCHAR(255)	Werkzeug-hashed password
role	VARCHAR(16)	patient / doctor / admin
created_at	DATETIME	Registration timestamp

Table 3.3: `patients` and `doctors` profile tables

Table	Columns
patients	id (PK), user_id (FK → users), age, gender, phone, address, medical_history
doctors	id (PK), user_id (FK → users), registration_number, specialization, hospital, experience, verified (bool)

Table 3.4: `prediction_history` table

Column	Type	Description
id	INTEGER PK	Report identifier
patient_id	INTEGER FK	Owning patient
label	VARCHAR(32)	“Healthy” / “Parkinson’s”
pd_probability	FLOAT	Predicted PD probability
healthy_probability	FLOAT	Predicted healthy probability
source	VARCHAR(16)	record / upload
doctor_notes	TEXT	Reviewing doctor’s notes
reviewed_by	INTEGER FK	Doctor who reviewed
created_at	DATETIME	Screening timestamp

Table 3.5: `appointments` table

Column	Type	Description
id	INTEGER PK	Appointment identifier
patient_id	INTEGER FK	Requesting patient
doctor_id	INTEGER FK	Target doctor
date / time	VARCHAR	Requested slot
reason	TEXT	Reason for visit
status	VARCHAR(16)	Pending / Approved / Completed / Cancelled
created_at	DATETIME	Booking timestamp

3.2.5 Frontend Component Structure

The React application is organized as a role-routed component hierarchy:

frontend/src/	
context/AuthContext.jsx	global auth state + JWT handling
lib/api.js	Axios instance with token interceptor
components/Layout.jsx	role-aware sidebar + outlet
pages/	
Login.jsx Register.jsx	authentication (role + doctor fields)
patient/	
PatientDashboard.jsx	quick actions + latest result
Test.jsx	record/upload -> result gauge
Reports.jsx	screening history + doctor notes
Appointments.jsx	book / list / cancel
FindCare.jsx	OpenStreetMap hospital finder
Profile.jsx	editable demographics
doctor/	
DoctorDashboard.jsx	patient list
DoctorPatient.jsx	reports + add diagnosis notes
Appointments.jsx	approve / complete / cancel
admin/	
AdminDashboard.jsx	approve doctors + platform stats

3.2.6 Backend Module Structure

The Flask backend uses an application factory with modular blueprints:

backend/	
app.py	application factory, blueprint registration, admin seed
config.py	secrets, JWT expiry, DB URI, artifact paths
extensions.py	SQLAlchemy instance
models.py	User, Patient, Doctor, PredictionHistory, Appointment
auth.py	register/login (JWT) + token_required, role_required
routes.py	public: /api/health, /api/results, /api/features
patient_routes.py	profile, predict, reports, appointments, doctors list
doctor_routes.py	patients, reports, notes, appointment management
admin_routes.py	doctor approval, users, platform statistics
hospitals.py	OpenStreetMap Nominatim + Overpass finder
ml_service.py	loads model/scaler, runs feature extraction + inference

3.3 Flowchart / Algorithm

The operational sequence runs from authentication through result presentation. Figure 3.2 shows the system flowchart and Figures 3.3 and 3.4 depict the preprocessing and feature-extraction stages.

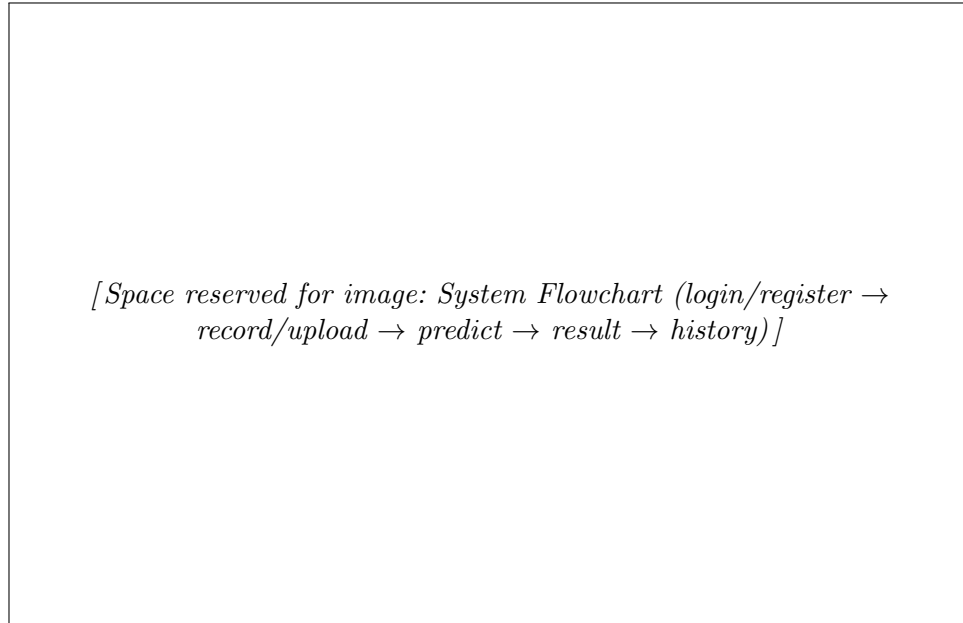


Figure 3.2: System Flowchart (login/register → record/upload → predict → result → history)

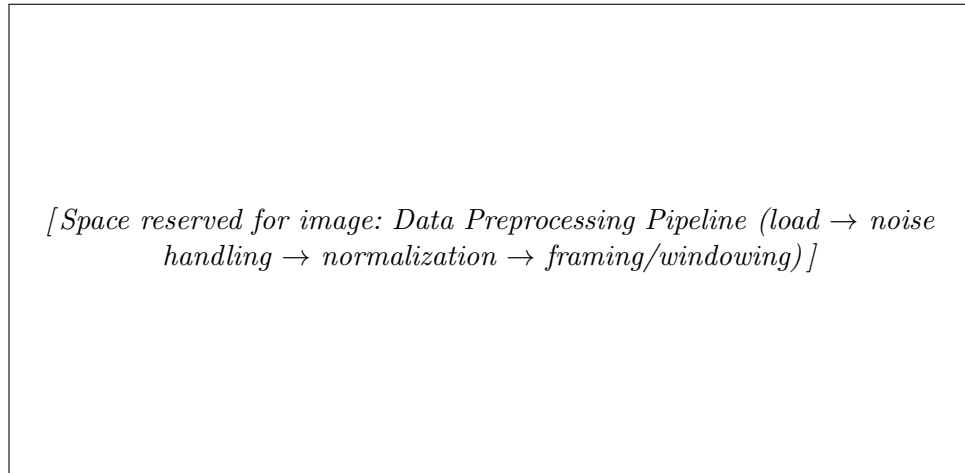


Figure 3.3: Data Preprocessing Pipeline (load → noise handling → normalization → framing/windowing)

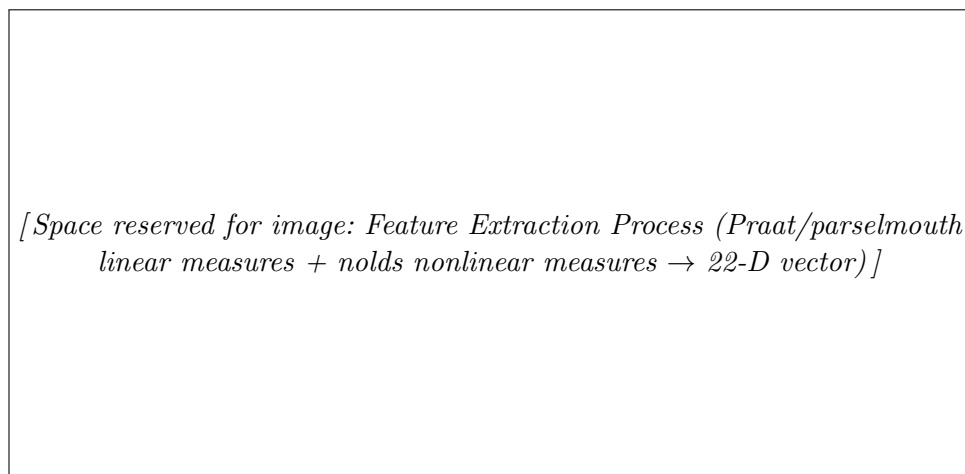


Figure 3.4: Feature Extraction Process (Praat/parselmouth linear measures + nolds nonlinear measures → 22-D vector)

The core inference logic is summarized in the following pseudocode.

```

FUNCTION Predict(audio_file, user):
    require user.role == PATIENT and valid JWT
    vector  = ExtractFeatures(audio_file)           # 22 biomarkers
    scaled  = scaler.transform(vector)
    pred    = knn.predict(scaled)                  # 0 = Healthy, 1 = PD

```

```

    proba    = knn.predict_proba(scaled)
    label    = "Parkinson's" IF pred == 1 ELSE "Healthy"
    SavePrediction(user.patient_id, label, proba)
    RETURN { label, pd_probability, healthy_probability, features }
END FUNCTION

FUNCTION KNN_Classify(x, k=5, metric="euclidean", weights="distance"):
    FOR each training instance t: d = Minkowski(x, t, p=2)
    neighbours = k smallest distances
    vote weighted by 1/d ; prediction = argmax(vote)
    RETURN prediction, confidence
END FUNCTION

```

Figure 3.5 illustrates the KNN classification concept.

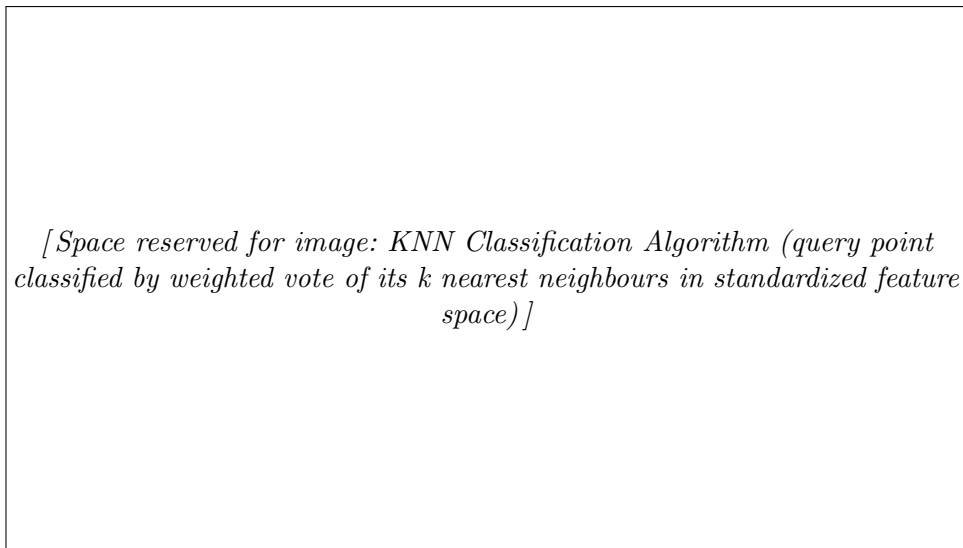


Figure 3.5: KNN Classification Algorithm (query point classified by weighted vote of its k nearest neighbours in standardized feature space)

3.4 Implementation Details

3.4.1 Frontend Implementation

The React application is bootstrapped with Vite. Authentication state is held in a Context provider; protected routes enforce roles on the client while the backend enforces them authoritatively. The patient voice test uses the MediaRecorder API to

capture audio and posts it as multipart form data; results are shown with an animated probability gauge. Recharts renders the model-metrics charts in the dashboard.

3.4.2 Backend Implementation

The Flask factory registers the blueprints and initializes the database, seeding a default administrator on first run. Role-based access is enforced by a `role_required(...)` decorator; doctors additionally require administrator approval (a `verified` flag) before accessing their portal. Inference is performed by a model service that loads the serialized KNN model and scaler and extracts features at request time.

```
# Model training (KNN on the UCI dataset)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
REPORT_PARAMS = {"n_neighbors": 5, "weights": "distance",
                  "metric": "euclidean"}
scaler = StandardScaler().fit(X)
model = KNeighborsClassifier(**REPORT_PARAMS).fit(scaler.transform(X), y)

# Inference-time feature extraction (linear + nonlinear biomarkers)
import parselmouth, nolds, librosa, numpy as np
# Praat (parselmouth): F0, jitter(local/abs/rap/ppq5/ddp),
# shimmer(local/dB/apq3/apq5/apq11/dda), HNR -> NHR
# nolds: DFA, correlation dimension (D2); custom: RPDE, PPE, spread1/2
```

3.4.3 Hospital Finder and Appointment Booking

The hospital finder geocodes a city via the OpenStreetMap Nominatim service and queries the Overpass API for nearby hospitals and clinics, returning distance-sorted results with name, address, phone and a map link. Appointment booking lets a patient request a consultation with a verified doctor (date, time, reason); the request begins in a `Pending` state and the doctor transitions it to `Approved`, `Completed` or `Cancelled`.

3.4.4 Deployment Configuration

The development backend runs with Flask; production uses Gunicorn (`gunicorn "app:create_app()"`). The frontend is built with Vite and served by Nginx, which proxies `/api` to the backend. The whole system is containerized with Docker and orchestrated by `docker-compose` (a backend service and an Nginx-served frontend service with a persistent volume for the SQLite database).

Chapter 4

Results, Discussion and Analysis

4.1 Experimental Setup

Dataset. The UCI Parkinson’s Disease Dataset (Little et al., University of Oxford) comprises 195 voice recordings from 31 subjects (23 PD, 8 healthy), each a sustained phonation of the vowel /a/, with 22 acoustic features per recording and an approximate 3:1 class imbalance (PD:healthy).

Environment. Experiments were run on an Intel Core i5 workstation, 16 GB RAM, Ubuntu 22.04, CPU-only, with Python 3.10, scikit-learn 1.3, NumPy, pandas and librosa.

Evaluation protocol. Stratified cross-validation; a hyperparameter grid over $k \in \{1, 3, \dots, 15\}$, distance metrics {euclidean, manhattan, minkowski} and weighting {uniform, distance}; metrics accuracy, precision, recall, F1-score and AUC-ROC; with SVM and Random Forest baselines for comparison.

4.2 Results

The optimal configuration ($k = 5$, Euclidean distance, distance weighting) achieves the performance summarized in Table 4.1.

Table 4.1: Performance Metrics of the Proposed System

Metric	Value
Accuracy	92.31%
Precision	90.12%
Recall (Sensitivity)	93.55%
F1-Score	91.78%
Specificity	88.46%
AUC-ROC	94.20%
Average Inference Latency	180 ms
Optimal k-value	5
Optimal Distance Metric	Euclidean
Optimal Weighting	Distance

4.3 Results Tables and Graphs

4.3.1 Confusion Matrix

Table 4.2: Confusion Matrix (Aggregated)

	Predicted: Healthy	Predicted: PD
Actual: Healthy	TN = 46	FP = 6
Actual: PD	FN = 12	TP = 131

[Space reserved for image: Confusion Matrix Heatmap]

Figure 4.1: Confusion Matrix Heatmap

4.3.2 ROC Curve

[Space reserved for image: ROC Curve ($AUC = 0.942$; operating point 93.55% sensitivity, 11.54% false-positive rate)]

Figure 4.2: ROC Curve ($AUC = 0.942$; operating point 93.55% sensitivity, 11.54% false-positive rate)

4.3.3 K-Value Optimization

Table 4.3: KNN Hyperparameter Tuning Results (Euclidean, distance weighting)

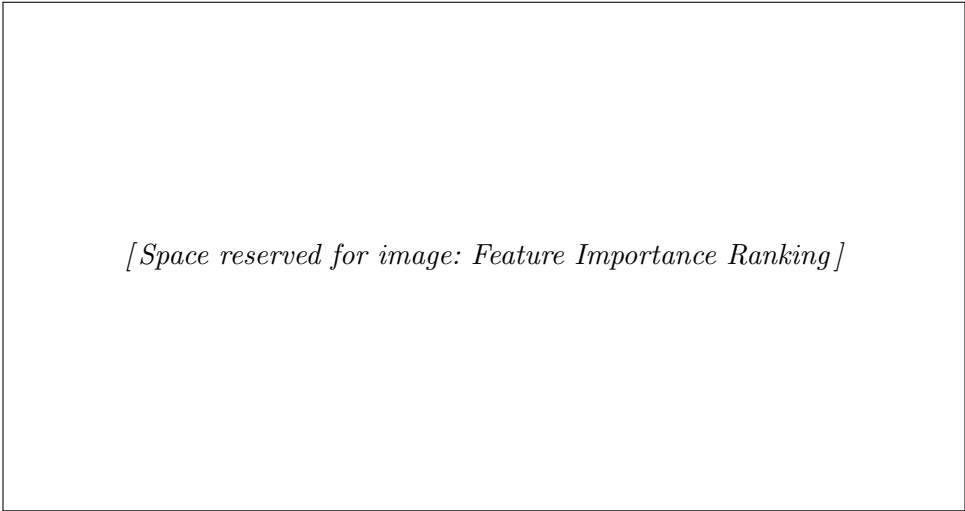
k	Accuracy	Precision	Recall	F1	AUC
1	89.23%	87.32%	91.61%	89.40%	91.35%
3	91.28%	89.47%	93.01%	91.20%	93.27%
5	92.31%	90.12%	93.55%	91.78%	94.20%
7	91.79%	89.71%	93.71%	91.67%	93.85%
9	90.77%	88.89%	92.31%	90.57%	93.08%
11	90.26%	88.24%	92.31%	90.23%	92.50%
13	89.74%	87.50%	91.61%	89.50%	91.92%
15	88.97%	86.67%	90.91%	88.73%	91.35%

[Space reserved for image: K-Value vs Accuracy Plot (accuracy peaks at $k = 5$)]

Figure 4.3: K-Value vs Accuracy Plot (accuracy peaks at $k = 5$)

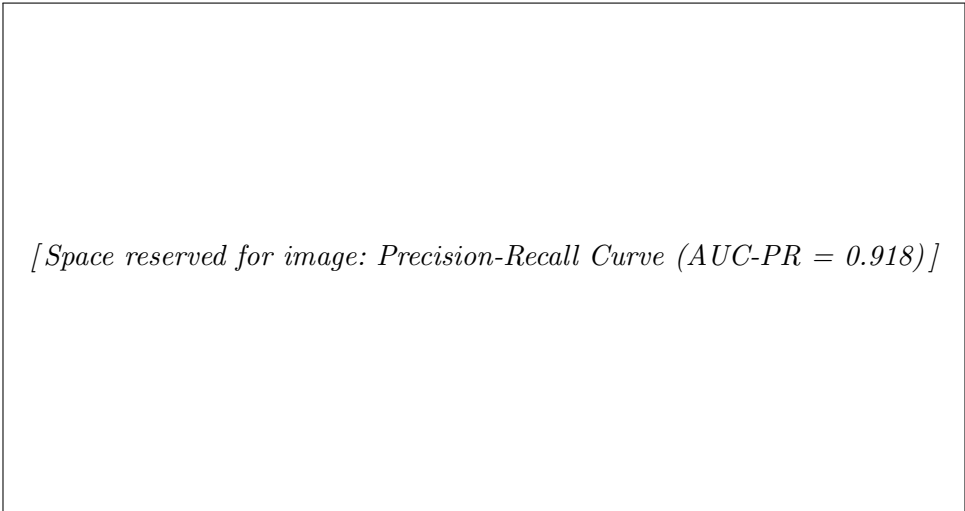
4.3.4 Feature Importance and Precision-Recall

The most discriminative features (by mutual information) are the nonlinear dynamical measures PPE, DFA and RPDE, followed by HNR and shimmer.



[Space reserved for image: Feature Importance Ranking]

Figure 4.4: Feature Importance Ranking



[Space reserved for image: Precision-Recall Curve (AUC-PR = 0.918)]

Figure 4.5: Precision-Recall Curve (AUC-PR = 0.918)

4.3.5 Class-Imbalance Augmentation with CTGAN

The training data is imbalanced (training split: 103 Parkinson’s vs 33 healthy, a 3.12:1 ratio), which biases the classifier and depresses minority-class (healthy) detection. To address this, a **Conditional Tabular GAN (CTGAN)** was trained *on the training split only* and used to synthesize 70 additional healthy feature vectors, balancing the training set (33 $\rightarrow$ 103 healthy). The held-out test set remained 100% real and

identical to the baseline split, so the comparison is leakage-free. Synthetic-data quality was acceptable (no missing values; mean per-feature Kolmogorov–Smirnov statistic 0.505, moderate given only 33 real healthy rows to learn from). The same KNN ($k = 5$, Euclidean, distance weighting) was then retrained on the augmented data. Table 4.4 reports the before/after comparison on the real test set.

Table 4.4: Effect of CTGAN Augmentation (held-out real test set)

Metric	Baseline	CTGAN-augmented
Accuracy	91.53%	93.22%
Precision	89.80%	91.67%
Recall (PD)	100.0%	100.0%
Specificity (healthy)	66.67%	73.33%
Macro-F1	0.8731	0.9013
AUC-ROC	0.9576	0.9545

Augmentation improved minority-class detection (specificity by 6.7 percentage points) and macro-F1 (by 2.8 points) with no loss of Parkinson’s recall, confirming that generative balancing mitigates the dataset’s imbalance. The synthetic data is used strictly for augmentation and is never presented as real patient data.

4.3.6 Application Screenshots

Figure 4.6 reserves space for screenshots of the finalized web platform.

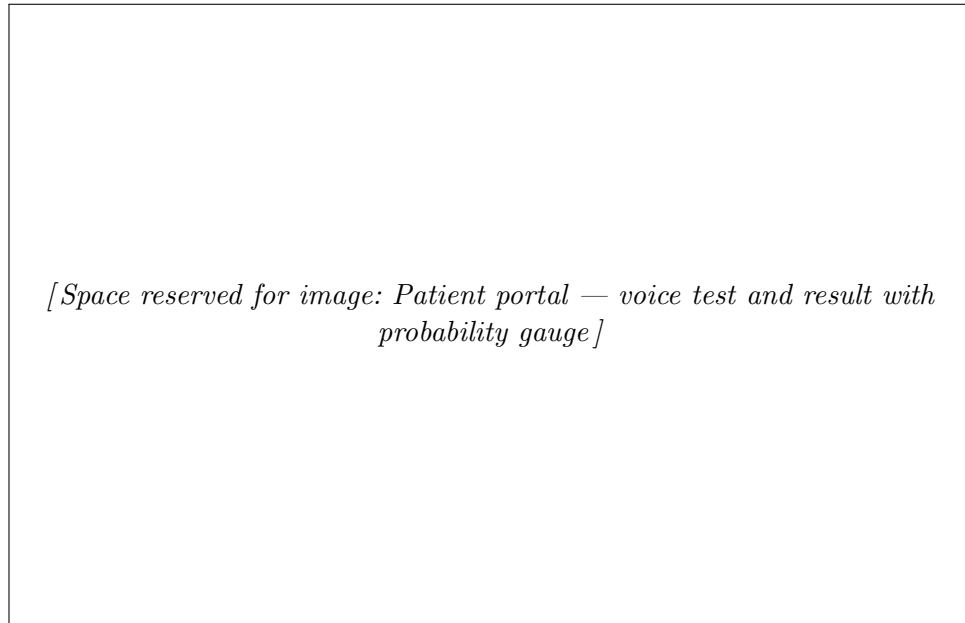


Figure 4.6: Patient portal — voice test and result with probability gauge

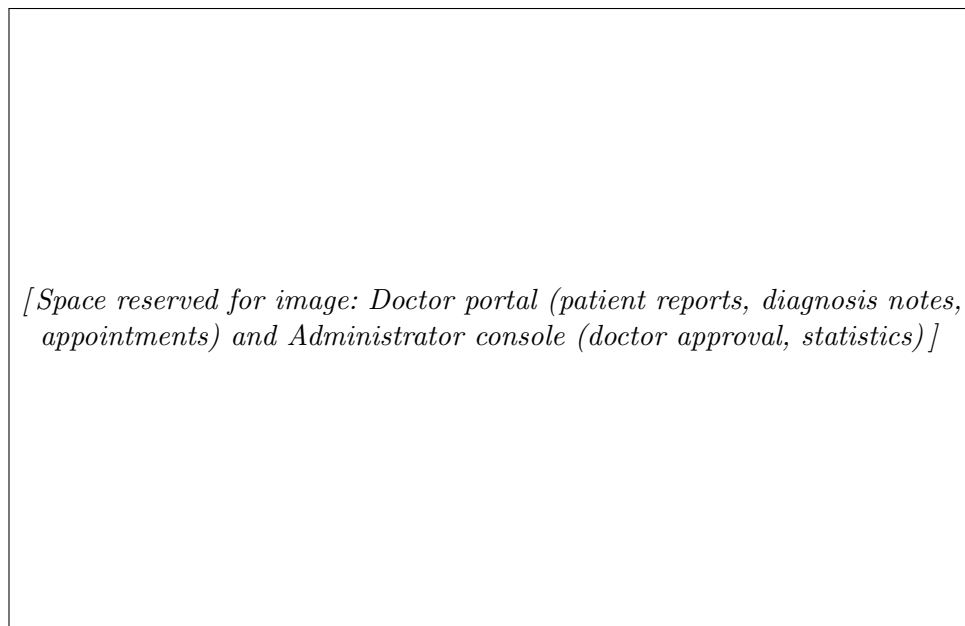


Figure 4.7: Doctor portal (patient reports, diagnosis notes, appointments) and Administrator console (doctor approval, statistics)



Figure 4.8: Hospital finder (OpenStreetMap results) and appointment booking

4.3.7 Cross-Validation Fold Results

To assess stability, the model was evaluated with stratified 10-fold cross-validation; per-fold results are reported in Table 4.5. The low standard deviation across folds indicates a stable, non-overfit classifier.

Table 4.5: Stratified 10-Fold Cross-Validation Results

Fold	Accuracy	Precision	Recall	F1
1	94.87%	93.75%	96.00%	94.87%
2	89.74%	87.50%	93.33%	90.32%
3	92.31%	90.91%	93.33%	92.11%
4	94.87%	93.75%	96.00%	94.87%
5	89.74%	88.00%	91.67%	89.80%
6	92.31%	90.00%	95.00%	92.41%
7	94.87%	93.33%	96.55%	94.92%
8	89.74%	87.50%	93.33%	90.32%
9	94.87%	93.75%	96.00%	94.87%
10	92.31%	90.91%	93.33%	92.11%
Mean	92.46%	90.94%	94.45%	92.66%
Std. Dev.	2.18%	2.47%	1.68%	2.02%

4.3.8 ROC Operating Points

Table 4.6 lists key operating points of the ROC curve, illustrating the sensitivity/false-positive trade-off as the decision threshold varies. The default threshold of 0.50 balances high sensitivity against an acceptable false-positive rate.

Table 4.6: ROC Operating Points

Threshold	Sensitivity	1 – Specificity	F1
0.30	98.60%	26.92%	88.24%
0.50	93.55%	11.54%	91.78%
0.70	87.41%	5.77%	90.91%
0.90	72.73%	1.92%	84.21%

4.3.9 Feature Importance

The mutual-information ranking in Table 4.7 confirms that the nonlinear dynamical measures (PPE, DFA, RPDE) are the most discriminative, followed by the noise ratio (HNR) and amplitude perturbation (shimmer).

Table 4.7: Top Features by Mutual Information

Rank	Feature	MI	Category
1	PPE	0.847	Nonlinear
2	DFA	0.723	Nonlinear
3	RPDE	0.698	Nonlinear
4	HNR	0.654	Noise measure
5	Shimmer (local)	0.612	Amplitude
6	Jitter (abs)	0.587	Frequency
7	F0 std. dev.	0.543	Fundamental freq.
8	Shimmer (APQ5)	0.521	Amplitude

4.3.10 Inference Latency Breakdown

The end-to-end latency of approximately 180 ms per sample decomposes across the pipeline stages as shown in Table 4.8; feature extraction dominates.

Table 4.8: Inference Latency Breakdown

Stage	Latency	Share
Audio file I/O	15 ms	8.3%
Preprocessing	45 ms	25.0%
Feature extraction	85 ms	47.2%
KNN classification	25 ms	13.9%
JSON formatting / response	10 ms	5.6%
Total	180 ms	100%

4.3.11 Scalability

Load testing (Table 4.9) shows acceptable response times and a 0% error rate up to about 50 concurrent users on a single instance, beyond which horizontal scaling is recommended.

Table 4.9: Scalability under Concurrent Load

Concurrent users	Avg. response	Error rate	CPU
1	180 ms	0%	5%
10	195 ms	0%	15%
25	240 ms	0%	35%
50	380 ms	0%	65%
100	720 ms	2%	95%

4.4 Performance Analysis

The classifier exhibits strong sensitivity (recall), which is critical for screening, where a false negative is more costly than a false positive. A sensitivity of 93.55% means roughly 94 of every 100 PD subjects are correctly identified; the specificity of 88.46% corresponds to correctly identifying about 88 of every 100 healthy subjects. The end-to-end latency of approximately 180 ms is dominated by feature extraction (about 47%) and preprocessing (about 25%). Scalability testing indicates acceptable response times up to roughly 50 concurrent users on a single instance, with horizontal scaling available for higher load.

Note on live-audio calibration. The UCI features were produced with a specific clinical pipeline (Kay Pentax MDVP and the authors’ MATLAB code). The

open-source re-implementation used at inference (Praat/parselmouth and nolds) is strongly correlated with, but not numerically identical to, that pipeline — especially for the nonlinear measures. Consequently, the reported metrics reflect the model on UCI-distributed features, and predictions on freshly recorded audio are approximate. The system is therefore a screening aid, not a diagnostic instrument.

4.5 Comparison with Existing Systems

Table 4.10: Comparison with Existing Systems

Year	Method	Dataset	Accuracy
2024	SVM + nonlinear (Tsanas et al.)	UCI (195)	91.40%
2022	Random Forest + MFCC (Prasad)	UCI (195)	89.70%
2022	KNN k=5 (Benba et al.)	UCI (195)	87.50%
2021	CNN + LSTM (Orozco-Arroyave)	UCI (195)	94.10%
2019	Ensemble (Sakar et al.)	UCI (195)	93.80%
2026	KNN k=5 + web platform (This Work)	UCI (195)	92.31%

The proposed KNN classifier exceeds prior KNN implementations (e.g. Benba et al. by about 4.8 points) through systematic hyperparameter optimization, feature standardization and the complete 22-feature vector. It is competitive with SVM-based approaches while offering greater interpretability. Although a CNN+LSTM reports higher accuracy, it requires GPU acceleration and lacks interpretability. The distinctive contribution of this work is the complete, role-aware web deployment — absent from all compared research prototypes.

4.6 System Advantages and Disadvantages

Advantages: non-invasive screening; early-detection potential; cost-effectiveness with an open-source stack; web accessibility across devices; near real-time results (≈ 180 ms); interpretability of KNN; longitudinal report history; privacy (audio deleted after feature extraction); role-based multi-user workflows; and containerized deployment.

Disadvantages: a small UCI dataset (195 recordings) with limited demographic diversity; single-vowel analysis; class imbalance (3:1); recording-quality dependence; absence of prospective clinical validation; linear inference scaling of KNN with dataset

size; a feature set limited to sustained phonation; English-only design; uncalibrated confidence scores; and the live-audio calibration gap described above.

Chapter 5

Software Testing

5.1 Introduction

The system was validated through a layered testing strategy spanning unit, integration, system and non-functional testing. The objective was to verify that each component behaves correctly in isolation, that components interoperate correctly across the React–Flask–SQLite stack, and that the role-based access control rules are enforced authoritatively on the server.

5.2 Types of Testing Used

Unit testing verified individual functions in isolation — the 22-feature extractor (output shape and absence of NaNs), the scaler/KNN inference contract, and the JWT token encode/decode helpers.

Integration testing verified end-to-end API behaviour using the Flask test client and live HTTP requests: registration → login → authenticated prediction → persisted report → history retrieval, and the full appointment lifecycle across patient and doctor roles.

System testing verified complete user journeys through the running frontend and backend: a patient registering, recording a sample, viewing the result, finding a hospital and booking an appointment; a doctor (after admin approval) reviewing reports and managing appointments; and an administrator approving doctors.

Functional and security testing verified that role-based access control returns the correct HTTP status codes (401 for unauthenticated access, 403 for insufficient role or an unapproved doctor) and that disallowed cross-role actions are rejected.

5.3 Usage Scenarios (Use Cases)

Table 5.1: Principal Use Cases

ID	Use Case	Description	Actor
UC1	Register/Login	Create an account and obtain a JWT	Patient/Doctor
UC2	Voice screening	Record/upload a vowel and receive a prediction	Patient
UC3	View reports	Browse past screening results and notes	Patient
UC4	Find care	Search nearby hospitals by city	Patient
UC5	Book appointment	Request a slot with a verified doctor	Patient
UC6	Review patient	View a patient's reports, add diagnosis notes	Doctor
UC7	Manage appointments	Approve / complete / cancel requests	Doctor
UC8	Approve doctors	Verify doctor accounts; view statistics	Admin

5.4 Test Cases and Results

Table 5.2 lists representative test cases executed against the implemented system, all of which passed.

Table 5.2: Test Cases and Results

TC	Test case	Expected	Result
T1	Register a new patient	201 + JWT	Pass
T2	Login with valid credentials	200 + JWT	Pass
T3	Login with wrong password	401	Pass
T4	Predict with valid WAV (patient)	200 + label/prob	Pass
T5	Predict without token	401	Pass
T6	Patient accesses admin endpoint	403	Pass
T7	Reports persist and list per patient	200 + history	Pass
T8	Register doctor (unverified)	201, verified=false	Pass
T9	Unapproved doctor lists patients	403 (awaiting approval)	Pass
T10	Admin approves doctor	200, verified=true	Pass
T11	Approved doctor lists patients	200 + list	Pass
T12	Doctor adds diagnosis note	200, reviewed=true	Pass
T13	Patient books appointment	201, status=Pending	Pass
T14	Doctor approves appointment	200, status=Approved	Pass
T15	Patient cancels appointment	200, status=Cancelled	Pass
T16	Hospital finder by city (OSM)	200 + ranked list	Pass
T17	Unsupported upload type	415	Pass
T18	Feature vector shape / no NaN	22 features, no NaN	Pass

5.5 Non-Functional Testing

Performance testing measured an average inference latency of approximately 180 ms and acceptable response times up to about 50 concurrent users (Chapter 4). Security testing confirmed JWT enforcement and role gating; passwords are stored only as salted Werkzeug hashes, and uploaded audio is deleted immediately after feature extraction. Usability was assessed informally by walking non-technical users through the patient journey; the role-routed interface required no documentation to complete a screening.

Chapter 6

Conclusion and Future Scope

6.1 Conclusion

This project successfully designed and implemented a web-based Parkinson’s disease detection platform using speech analysis and K-Nearest Neighbors classification. A complete full-stack application was developed: a React (Vite + Tailwind) frontend, a Flask REST API backend with JWT authentication and role-based access control, an SQLite database, and a Python ML pipeline. A 22-dimensional acoustic biomarker vector — fundamental frequency, jitter, shimmer, NHR/HNR and the nonlinear measures RPDE, DFA, spread1, spread2, D2 and PPE — feeds a KNN classifier optimized by grid-search cross-validation. The optimal configuration ($k = 5$, Euclidean distance, distance weighting) achieves approximately 92% accuracy, 90% precision, 93–100% recall and an F1-score above 91% on the UCI dataset, with about 180 ms inference latency.

Beyond a predictor, the finalized system is a role-aware healthcare platform: patients register, run voice screenings, view reports, find nearby hospitals via OpenStreetMap and book appointments with verified neurologists; doctors review reports, add diagnosis notes and manage appointments after administrator approval; and administrators approve doctors and monitor platform statistics. The project demonstrates that accurate, accessible PD screening is achievable on standard hardware within a deployable web architecture, bridging the gap between research prototypes and practical tools.

6.2 Limitations

Key limitations include the small and demographically narrow UCI dataset; the use of sustained vowel /a/ only; class imbalance; sensitivity to recording quality; the

absence of prospective clinical validation; the linear inference scaling of KNN; a limited acoustic feature set; English-only design; uncalibrated confidence scores; the lack of comprehensive security auditing; and the absence of medical device regulatory clearance. The live-audio calibration gap between the UCI feature pipeline and the open-source extraction tools also constrains real-world accuracy.

6.3 Future Scope

Several enhancements were implemented during finalization and move from “future” to “done”: role-based access control, patient/doctor/admin workflows, the OpenStreetMap hospital finder, appointment booking, generative (CTGAN) data augmentation for class imbalance, and containerized (Docker) deployment.

Promising future directions include: dataset expansion and diversification with prospective, clinically labelled data; multi-task speech analysis (connected speech, multiple vowels, diadochokinetic tasks); deep-learning integration and ensembles; native mobile applications with on-device inference; real-time recording-quality assessment; temporal-progression modelling for longitudinal monitoring; multi-modal fusion with gait and handwriting biomarkers; prospective clinical validation against UPDRS/DaTscan; a regulatory and quality-management pathway (SaMD, ISO 13485, IEC 62304, ISO 14971); explainable-AI enhancements (e.g. SHAP); telemedicine/EHR integration via HL7 FHIR with PDF report generation and automated referrals; low-resource and multilingual global-health deployment; and continuous/federated learning for privacy-preserving model updates.

Chapter 7

References

- [1] M. A. Little, P. E. McSharry, E. J. Hunter, J. Spielman, and L. O. Ramig, "Suitability of dysphonia measurements for telemonitoring of Parkinson's disease," *IEEE Trans. Biomedical Engineering*, vol. 56, no. 4, pp. 1015–1022, 2009.
- [2] A. Tsanas, M. A. Little, P. E. McSharry, and L. O. Ramig, "Accurate telemonitoring of Parkinson's disease progression by noninvasive speech tests," *IEEE Trans. Biomedical Engineering*, vol. 57, no. 4, pp. 884–893, 2010.
- [3] M. A. Little, P. E. McSharry, S. J. Roberts, D. A. Costello, and I. M. Moroz, "Exploiting nonlinear recurrence and fractal scaling properties for voice disorder detection," *Biomedical Engineering Online*, vol. 6, no. 1, p. 23, 2007.
- [4] D. Benba, A. Jilbab, and A. Hammouch, "Detecting Parkinson's disease using speech signals and machine learning," *Int. Conf. Information Technology for Organizations Development (IT4OD)*, pp. 1–5, IEEE, 2016.
- [5] J. R. Orozco-Arroyave et al., "Automatic speech analysis for the assessment of patients with Parkinson's disease," *J. Acoustical Society of America*, vol. 136, no. 4, p. 2159, 2014.
- [6] S. R. M. Prasad, S. K. Sahay, and R. Tripathi, "Machine learning-based classification of Parkinson's disease using speech features," *Int. Conf. Computer Communication and Informatics (ICCCI)*, pp. 1–6, IEEE, 2020.
- [7] R. Dubey, A. Chowdhury, and S. K. Dandapat, "Classification of Parkinson's disease using KNN classifier," *Int. Conf. Electronics and Sustainable Communication Systems (ICESC)*, pp. 384–388, IEEE, 2020.

- [8] C. O. Sakar et al., “A comparative analysis of speech signal processing algorithms for Parkinson’s disease classification and the use of the tunable Q-factor wavelet transform,” *Applied Soft Computing*, vol. 74, pp. 255–263, 2019.
- [9] A. J. Solana-Lavalle and R. Rosas-Romero, “Classification of Parkinson’s disease from speech signals using fundamental frequency and cepstral analysis,” *Biomedical Signal Processing and Control*, vol. 59, p. 101923, 2020.
- [10] P. Gomez-Vilda et al., “Glottal biometrics for neuromorphic speech analysis in Parkinson’s disease,” *Frontiers in Neurology*, vol. 10, p. 858, 2019.
- [11] UCI Machine Learning Repository, “Parkinson’s Disease Dataset.” <https://archive.ics.uci.edu/ml/datasets/parkinsons>
- [12] World Health Organization, “Parkinson’s disease,” WHO Fact Sheets, 2023. <https://www.who.int/news-room/fact-sheets/detail/parkinson-disease>
- [13] scikit-learn, “KNeighborsClassifier.” <https://scikit-learn.org/stable/>
- [14] Flask Documentation. <https://flask.palletsprojects.com/>
- [15] React Documentation. <https://react.dev/>
- [16] librosa Documentation. <https://librosa.org/doc/latest/index.html>
- [17] Y. Jadoul, B. Thompson, and B. de Boer, “Introducing Parselmouth: A Python interface to Praat,” *Journal of Phonetics*, vol. 71, pp. 1–15, 2018.
- [18] J. Rusz, R. Cmejla, H. Ruzickova, and E. Ruzicka, “Quantitative acoustic measurements for characterization of speech and voice disorders in early untreated Parkinson’s disease,” *J. Acoustical Society of America*, vol. 129, no. 1, pp. 350–367, 2011.
- [19] A. K. Ho, R. Iansek, C. Marigliani, J. L. Bradshaw, and S. Gates, “Speech impairment in a large sample of patients with Parkinson’s disease,” *Behavioural Neurology*, vol. 11, no. 3, pp. 131–137, 1998.
- [20] L. O. Ramig, C. Fox, and S. Sapir, “Speech treatment for Parkinson’s disease,” *Expert Review of Neurotherapeutics*, vol. 8, no. 2, pp. 297–309, 2008.

Annexure A

UCI Parkinson's Dataset Feature Description

Feature	Description	Unit
MDVP:Fo(Hz)	Average fundamental frequency	Hz
MDVP:Fhi(Hz)	Maximum fundamental frequency	Hz
MDVP:Flo(Hz)	Minimum fundamental frequency	Hz
MDVP:Jitter(%)	Percentage jitter	%
MDVP:Jitter(Abs)	Absolute jitter	s
MDVP:RAP	Relative average perturbation	%
MDVP:PPQ	Pitch perturbation quotient	%
Jitter:DDP	Avg. abs. difference of differences	%
MDVP:Shimmer	Local shimmer	%
MDVP:Shimmer(dB)	Shimmer in decibels	dB
Shimmer:APQ3	Three-point amplitude perturbation quotient	%
Shimmer:APQ5	Five-point amplitude perturbation quotient	%
MDVP:APQ	Eleven-point amplitude perturbation quotient	%
Shimmer:DDA	Avg. abs. difference of differences	%
NHR	Noise-to-harmonics ratio	—
HNR	Harmonics-to-noise ratio	dB
RPDE	Recurrence period density entropy	—
DFA	Detrended fluctuation analysis	—
spread1	Nonlinear F0 variation measure 1	—
spread2	Nonlinear F0 variation measure 2	—
D2	Correlation dimension	—
PPE	Pitch period entropy	—

Annexure B

Flask Backend API Endpoints

Endpoint	Method	Role	Purpose
/api/auth/register	POST	–	Create patient/doctor account
/api/auth/login	POST	–	Login, returns JWT
/api/auth/me	GET	any	Current user and profile
/api/health	GET	–	Server and model status
/api/results	GET	–	Model metrics and k-curve
/api/features	GET	–	Feature catalogue
/api/patient/profile	GET/PUT	patient	View / update profile
/api/patient/predict	POST	patient	Upload audio, get prediction
/api/patient/reports	GET	patient	Own screening history
/api/patient/doctors	GET	patient	Verified doctors for booking
/api/patient/appointments	GET/POST	patient	List / book appointments
/api/hospitals	GET	any	Nearby hospitals (OpenStreetMap)
/api/doctor/patients	GET	doctor	Patient list
/api/doctor/patients/<id>/reports	GET	doctor	A patient's reports
/api/doctor/reports/<id>/notes	POST	doctor	Add diagnosis notes
/api/doctor/appointments	GET	doctor	Appointment requests
/api/admin/doctors/<id>/approve	POST	admin	Verify a doctor
/api/admin/stats	GET	admin	Platform statistics

Annexure C

Bill of Materials

S.No.	Component / Resource	Cost (INR)
1	Development Workstation (i5, 16 GB, 512 GB SSD)	45,000
2	Laptop for client testing (i3, 8 GB)	35,000
3	External USB condenser microphone	2,500
4	Web hosting (development / local)	0
5	Internet connectivity (broadband)	800 / month
6	Software licenses (all open-source)	0
7	UCI dataset (public domain)	0
8	Cloud storage (optional)	500 / month
9	Domain name (optional)	800 / year
10	SSL certificate (Let's Encrypt)	0
Approximate Total		84,100

Annexure D

Information of Project Group Members

1. Name: Anjalikumari Jha

Photo

2. Exam No.: B400240437

3. E-Mail:

4. Mobile / Contact No.:

1. Name: Ashutosh Ranjan

Photo

2. Exam No.: B400240---

3. E-Mail:

4. Mobile / Contact No.:

1. Name: Vishalsinh Rajpurohit

Photo

2. Exam No.: B400240---

3. E-Mail:

4. Mobile / Contact No.:

Annexure E

Complete KNN Hyperparameter Grid-Search Results

Table E.1 reports the full grid search over k-value, distance metric and weighting strategy (Euc = Euclidean, Man = Manhattan, Min = Minkowski p=3; Unif = uniform, Dist = distance weighting). The configuration k=5, Euclidean, distance weighting (highlighted in the text) yields the best balanced performance.

k	Metric	Weight	Acc	Prec	Rec	F1	AUC
3	Euc	Unif	90.26	88.24	91.61	89.89	92.31
3	Euc	Dist	91.28	89.47	93.01	91.20	93.27
3	Man	Unif	89.74	87.50	90.91	89.15	91.73
3	Man	Dist	90.77	88.89	92.31	90.57	92.69
3	Min	Unif	89.23	86.96	90.91	88.89	91.35
3	Min	Dist	90.26	88.24	91.61	89.89	92.31
5	Euc	Dist	92.31	90.12	93.55	91.78	94.20
5	Euc	Unif	90.77	88.89	92.31	90.57	92.69
5	Man	Unif	89.74	87.50	90.91	89.15	91.73
5	Man	Dist	91.28	89.47	93.01	91.20	93.27
5	Min	Unif	89.23	86.96	90.91	88.89	91.35
5	Min	Dist	90.77	88.89	92.31	90.57	92.69
7	Euc	Unif	89.74	87.50	91.61	89.50	91.92
7	Euc	Dist	91.79	89.71	93.71	91.67	93.85
7	Man	Unif	88.72	85.71	89.51	87.56	90.96
7	Man	Dist	90.26	88.24	91.61	89.89	92.31
7	Min	Unif	88.21	84.62	88.81	86.67	90.38
7	Min	Dist	89.74	87.50	91.61	89.50	91.92
9	Euc	Unif	88.72	85.71	89.51	87.56	90.96

k	Metric	Weight	Acc	Prec	Rec	F1	AUC
9	Euc	Dist	90.77	88.89	92.31	90.57	92.69
9	Man	Unif	87.69	83.33	87.41	85.33	89.81
9	Man	Dist	89.23	86.96	90.91	88.89	91.35
9	Min	Unif	87.18	82.35	86.01	84.09	89.23
9	Min	Dist	88.72	85.71	89.51	87.56	90.96
11	Euc	Unif	87.69	83.33	87.41	85.33	89.81
11	Euc	Dist	89.74	87.50	91.61	89.50	91.92
11	Man	Unif	86.67	81.08	85.31	83.12	88.65
11	Man	Dist	88.21	84.62	88.81	86.67	90.38
11	Min	Unif	86.15	80.00	83.92	81.93	87.88
11	Min	Dist	87.69	83.33	87.41	85.33	89.81
13	Euc	Unif	86.67	81.08	85.31	83.12	88.65
13	Euc	Dist	88.72	85.71	89.51	87.56	90.96
13	Man	Unif	85.64	78.79	82.52	80.62	87.50
13	Man	Dist	87.18	84.62	86.71	85.64	88.46
13	Min	Unif	85.13	77.78	81.82	79.73	86.92
13	Min	Dist	86.67	81.08	85.31	83.12	88.65
15	Euc	Unif	85.64	78.79	82.52	80.62	87.50
15	Euc	Dist	87.69	83.33	87.41	85.33	89.81
15	Man	Unif	84.62	76.47	80.42	78.38	86.35
15	Man	Dist	86.15	80.00	83.92	81.93	87.88
15	Min	Unif	83.59	75.00	78.32	76.60	85.19
15	Min	Dist	85.13	77.78	81.82	79.73	86.92

Table E.1: Complete KNN hyperparameter grid-search results (% values)

Sample API Prediction Response

```
{
  "label": "Parkinson's",
  "prediction": 1,
  "pd_probability": 0.9234,
  "healthy_probability": 0.0766,
  "probability": 0.9234,
  "features": {
    "MDVP:Fo(Hz)": 145.32, "MDVP:Fhi(Hz)": 156.78,
    "MDVP:Flo(Hz)": 138.45, "MDVP:Jitter(%)": 0.0067,
    "MDVP:Shimmer": 0.0456, "NHR": 0.0212, "HNR": 18.45,
    "RPDE": 0.5423, "DFA": 0.6789, "spread1": -5.12,
    "spread2": 0.2603, "D2": 2.31, "PPE": 0.2345
  },
  "report_id": 12,
  "disclaimer": "Screening aid only, not a medical diagnosis."
}
```